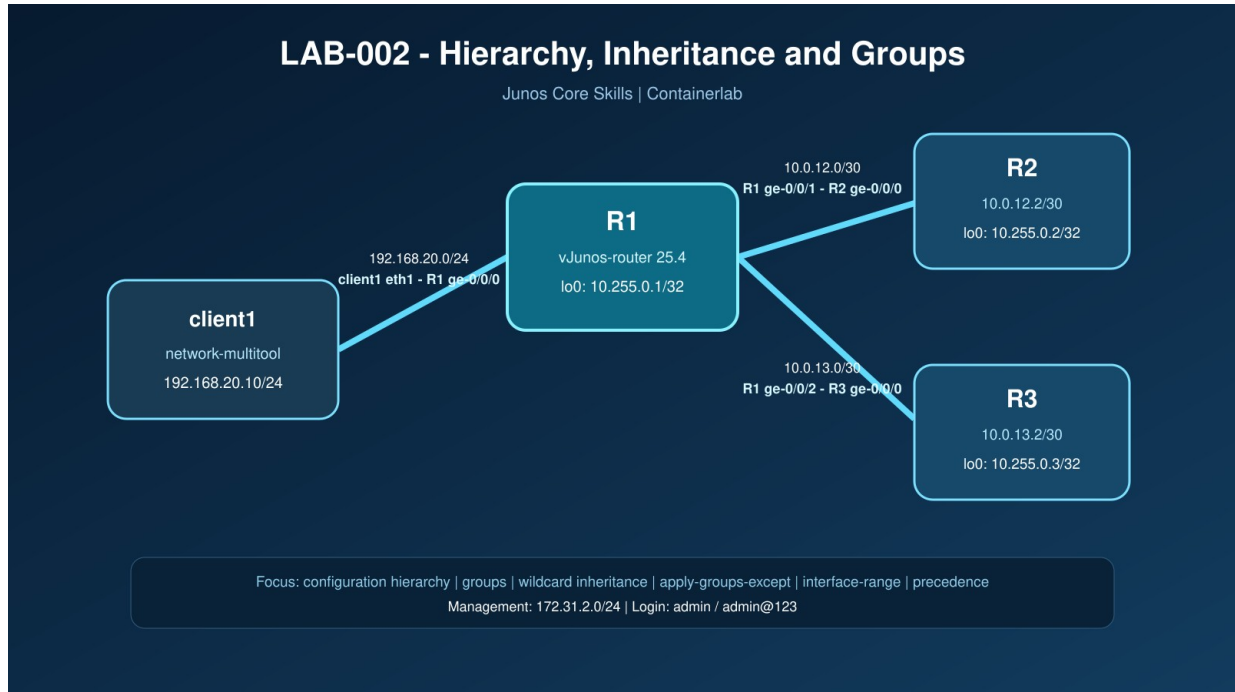


LAB-002 - Task & Solution Guide

Junos Core Skills | LAB-002 | Intermediate



Important: Use this guide only after attempting the task-only assessment. This is a self-contained guide: every section reproduces the complete task first, followed immediately by its step-by-step solution. Commands are shown in set format for precision; inspect the resulting hierarchy as part of every task.

Task 1 - Deploy and prove the baseline

Objective: Deploy the topology from a clean state and prove that the startup configuration and baseline forwarding are healthy.

TASK

Deploy the topology from a clean state. Prove that all startup configurations were loaded, all intended interface addresses and static routes exist, and ``scripts/verify.sh`` returns ``RESULT: PASS``.

Important: deployment output, ``show interfaces terse``, exact loopback routes, R1 exact static routes to R2/R3 loopbacks, and verification-script output.

SOLUTION

```
containerlab destroy -t lab002.clab.yml --cleanup
containerlab deploy -t lab002.clab.yml
chmod +x scripts/verify.sh
./scripts/verify.sh
```

On each router, log in as `admin` with password `admin@123` and verify:

```
show configuration system host-name
show interfaces terse
show route 10.255.0.0/24
show route protocol static
show system commit
```

On R1, explicitly prove the two startup static routes:

```
show route 10.255.0.2 exact
show route 10.255.0.3 exact
ping 10.255.0.2 source 10.255.0.1 count 5
ping 10.255.0.3 source 10.255.0.1 count 5
```

Explanation: Do not continue until the verification script prints `RESULT: PASS`.

Expected Result: All routers load their startup configuration, R1 reaches both remote loopbacks, and the verification script reports RESULT: PASS.

Task 2 - Navigate the Junos hierarchy efficiently

Objective: Navigate the Junos configuration hierarchy safely and distinguish candidate changes from the active configuration.

TASK

On R1, demonstrate at least two safe methods to move between configuration hierarchy levels. Add a candidate-only system location, inspect it from the relevant hierarchy and from the top, then discard it without affecting active configuration.

Important: hierarchy prompts, candidate diff, proof that active configuration did not change.

SOLUTION

```
configure
edit system
set location building JNCIE-LAB
show location
top
show system location
show | compare
rollback 0
show | compare
exit
show configuration system location
```

Explanation: `edit` enters a hierarchy, `up` moves one level, and `top` returns to the root. `rollback 0` reloads the active configuration into the candidate database, so the uncommitted location disappears.

Expected Result: The candidate diff becomes empty after rollback 0, and the active system location remains unchanged.

Task 3 - Build and apply a reusable system group

Objective: Create a reusable system group, apply it globally, and verify the effective inherited configuration.

TASK

On all routers, create a group named `SP-BASE` that supplies:

- a login message containing `AUTHORIZED ACCESS ONLY`;
- NETCONF over SSH;
- the time zone `Asia/Jakarta`.

Apply the group globally. Avoid duplicating these statements as explicit active configuration. Prove both the group definition and the inherited effective configuration.

Important: group definition, effective inherited configuration, comparison, validation, commit history.

SOLUTION

First remove duplicate explicit statements that the group will own, while keeping SSH untouched:

```
configure
delete system services netconf
delete system time-zone
set groups SP-BASE system login message "AUTHORIZED ACCESS ONLY"
set groups SP-BASE system services netconf ssh
set groups SP-BASE system time-zone Asia/Jakarta
set apply-groups SP-BASE
show | compare
commit check
commit comment "LAB-002 Task 3: apply reusable system baseline"
```

Verify the definition and effective configuration:

```
show configuration groups SP-BASE
show configuration system | display inheritance
show configuration system | display inheritance no-comments
show system commit
```

Explanation: Repeat on R2 and R3. `display inheritance` is essential because a normal configuration display can hide where the effective statements came from.

Expected Result: SP-BASE appears once as a group definition, while the effective system configuration shows the inherited login message, NETCONF, and time zone.

Task 4 - Create a router-specific exception

Objective: Exclude a group at a specific hierarchy without deleting the group, then restore normal inheritance.

TASK

R3 is a controlled exception and must not inherit `SP-BASE`. Use `apply-groups-except`; do not delete the group itself. Prove that R1 and R2 still inherit the group while R3 does not. Then restore inheritance on R3.

Important: exclusion configuration, effective configuration on all three routers, rollback or corrective diff, final verification.

SOLUTION

On R3:

```
configure
set system apply-groups-except SP-BASE
show | compare
commit check
commit comment "LAB-002 Task 4: temporarily exclude SP-BASE on R3"
run show configuration system | display inheritance
```

Compare R1/R2 with R3. The group still exists and remains globally applied, but inheritance below `system` is blocked on R3. Restore it:

```
delete system apply-groups-except SP-BASE
show | compare
commit check
commit comment "LAB-002 Task 4: restore SP-BASE inheritance on R3"
run show configuration system | display inheritance
```

Task 5 - Use a wildcard interface group

Objective: Use a wildcard group to apply a common MTU only to the intended core-facing interfaces.

TASK

On R1, create group `CORE-MTU` using a wildcard interface expression that targets only `ge-0/0/1` and `ge-0/0/2`. The group must supply an interface MTU of 2000. Apply it globally and prove:

- both core-facing interfaces inherit MTU 2000;
- the client-facing `ge-0/0/0` does not inherit it;
- existing addresses and reachability remain intact.

Important: wildcard group definition, inherited views of all three interfaces, interface operational MTU, `RESULT: PASS`.

SOLUTION

On R1, quote the wildcard expression so the CLI treats it as one token:

```
configure
set groups CORE-MTU interfaces "<ge-0/0/[1-2]>" mtu 2000
set apply-groups CORE-MTU
show | compare
commit check
commit comment "LAB-002 Task 5: inherit core interface MTU"
```

Verify all three interfaces, including the non-target:

```
show configuration groups CORE-MTU
show configuration interfaces ge-0/0/0 | display inheritance
show configuration interfaces ge-0/0/1 | display inheritance
show configuration interfaces ge-0/0/2 | display inheritance
show interfaces ge-0/0/1 extensive | match MTU
show interfaces ge-0/0/2 extensive | match MTU
```

Explanation: Then run `./scripts/verify.sh` from the host. The wildcard limits inheritance to core-facing interfaces; it must not modify `ge-0/0/0`.

Expected Result: R1 ge-0/0/1 and ge-0/0/2 inherit MTU 2000; ge-0/0/0 does not. Existing addressing and reachability remain intact.

Task 6 - Apply an interface-level exception

Objective: Apply a narrow interface-level exception while preserving inheritance on the other core link.

TASK

Exclude `CORE-MTU` only from R1 `ge-0/0/2`, leaving `ge-0/0/1` inherited. Prove the difference in effective and operational configuration. Restore the standard after completing the observation.

Important: before/after inheritance output, operational MTU, comparison, final restoration.

SOLUTION

```
configure
set interfaces ge-0/0/2 apply-groups-except CORE-MTU
show | compare
commit check
commit comment "LAB-002 Task 6: test MTU exception on R1 ge-0/0/2"
run show configuration interfaces ge-0/0/1 | display inheritance
run show configuration interfaces ge-0/0/2 | display inheritance
run show interfaces ge-0/0/1 extensive | match MTU
run show interfaces ge-0/0/2 extensive | match MTU
```

Restore the standard:

```
delete interfaces ge-0/0/2 apply-groups-except CORE-MTU
show | compare
commit check
commit comment "LAB-002 Task 6: restore inherited core MTU"
```

Explanation: Run the verification script again.

Expected Result: Only ge-0/0/2 temporarily loses the inherited CORE-MTU value; both core links return to the standard after restoration.

Task 7 - Compare explicit and interface-range configuration

Objective: Use an interface range for common configuration and prove precedence between explicit and expanded statements.

TASK

On R1, create interface range `CORE-LINKS` containing `ge-0/0/1` and `ge-0/0/2`. Use the range to supply the common description `SP-CORE-MEMBER`.

The startup configuration already provides the explicit descriptions `R1-TO-R2` and `R1-TO-R3`. Do not assume that the range description will replace them. Determine which source wins, then temporarily

remove the explicit description from `ge-0/0/1` and prove that the range description becomes effective. Restore the original description before completing the task.

Important: interface-range definition, inheritance views before and after removing the explicit statement, operational descriptions, your precedence conclusion, final clean configuration.

SOLUTION

```
configure
set interfaces interface-range CORE-LINKS member ge-0/0/1
set interfaces interface-range CORE-LINKS member ge-0/0/2
set interfaces interface-range CORE-LINKS description SP-CORE-MEMBER
show | compare
commit check
commit comment "LAB-002 Task 7: apply common core link description"
```

First inspect both member interfaces without changing their startup descriptions:

```
show configuration interfaces interface-range CORE-LINKS
show configuration interfaces ge-0/0/1 | display inheritance
show configuration interfaces ge-0/0/2 | display inheritance
show interfaces descriptions
```

Explanation: Both interfaces retain their explicit startup descriptions. Foreground configuration on a member interface takes priority over the value expanded from an interface range.

Temporarily remove the explicit description only from `ge-0/0/1`:

```
delete interfaces ge-0/0/1 description
show | compare
commit check
commit comment "LAB-002 Task 7: reveal inherited range description"
run show configuration interfaces ge-0/0/1 | display inheritance
run show interfaces descriptions | match ge-0/0/1
```

`ge-0/0/1` now uses `SP-CORE-MEMBER`, proving that the range value was present but previously overridden. Restore the production description while leaving `CORE-LINKS` available as a reusable membership object:

```
set interfaces ge-0/0/1 description R1-T0-R2
show | compare
commit check
commit comment "LAB-002 Task 7: restore explicit link description"
run show configuration interfaces ge-0/0/1 | display inheritance
run show interfaces descriptions
```

Explanation: The final operational descriptions are `R1-T0-R2` and `R1-T0-R3`. Their explicit values continue to override `SP-CORE-MEMBER` from `CORE-LINKS`.

Expected Result: Both explicit startup descriptions override the range value. Removing one explicit statement reveals SP-CORE-MEMBER, and restoring it returns the original operational description.

Task 8 - Prove precedence between overlapping interface ranges

Objective: Prove deterministic precedence between overlapping interface ranges without interfering with the CORE-MTU group.

TASK

On R1, create two interface ranges named `HOLD-A` and `HOLD-B`. Make `ge-0/0/1` a member of both ranges. Configure `hold-time up 1000` under `HOLD-A` and `hold-time up 2000` under `HOLD-B`.

Before validation, predict whether Junos will reject the overlap or resolve it through inheritance priority. Run `commit check`, commit the configuration, and prove the effective value. Then place `HOLD-B` before `HOLD-A` in configuration order and prove whether the effective value changes. Remove both test ranges when finished.

Do not modify MTU in this task; MTU remains owned by the `CORE-MTU` group from Task 5.

Important: initial range order, successful validation, inheritance output for both orders, your precedence conclusion, cleanup diff, final MTU inheritance, and `RESULT: PASS`.

SOLUTION

Create two overlapping ranges using `hold-time`, which is not configured by the startup configuration or `CORE-MTU`:

```
configure
set interfaces interface-range HOLD-A member ge-0/0/1
set interfaces interface-range HOLD-A hold-time up 1000
set interfaces interface-range HOLD-B member ge-0/0/1
set interfaces interface-range HOLD-B hold-time up 2000
show | compare
commit check
commit comment "LAB-002 Task 8: test overlapping range priority"
```

Junos accepts the overlap. Among interface ranges, the range defined first has the higher inheritance priority. Verify that `HOLD-A` initially supplies the effective value:

```
show configuration interfaces | display set | match "interface-range HOLD"
show configuration interfaces ge-0/0/1 | display inheritance
```

The effective statement should be `hold-time up 1000`, expanded from `HOLD-A`. Now change only the range order:

```
configure
insert interfaces interface-range HOLD-B before interface-range HOLD-A
show | compare
commit check
commit comment "LAB-002 Task 8: reverse overlapping range priority"
run show configuration interfaces | display set | match "interface-range HOLD"
run show configuration interfaces ge-0/0/1 | display inheritance
```


Explanation: With `HOLD-B` first, the effective statement should become `hold-time up 2000`. This demonstrates deterministic inheritance priority rather than a validation failure.

Remove the test ranges and prove that Task 5 still owns MTU:

```
delete interfaces interface-range HOLD-A
delete interfaces interface-range HOLD-B
show | compare
commit check
commit comment "LAB-002 Task 8: remove precedence test ranges"
run show configuration interfaces ge-0/0/1 | display inheritance
run show interfaces ge-0/0/1 extensive | match MTU
```

Explanation: The final interface must again show MTU 2000 inherited from `CORE-MTU`, and the host verification script must return `RESULT: PASS`.

Expected Result: The first-defined range supplies the effective hold-time; reversing range order changes the effective value. Cleanup restores MTU 2000 from CORE-MTU and forwarding remains healthy.

Final validation

```
show configuration groups
show configuration apply-groups
show configuration interfaces | display inheritance
show system commit
```

From the container host:

```
./scripts/verify.sh
```

The final state must have no unintended exclusions, no temporary `HOLD-A` or `HOLD-B` ranges, meaningful interface descriptions, inherited core MTU on both R1 core links, and `RESULT: PASS`.